

Projeto para UFCD 5425

Tema:

Sistema Integrado de Gestão para Servidores Discord de Roleplay

Objetivo Geral

Desenvolver uma aplicação em Python para administração de servidores Discord de Roleplay, permitindo gerir tickets, candidaturas, organizações, permissões e comunicação com a base de dados do servidor de jogo. O projeto utilizará Python, a biblioteca `discord.py` e bases de dados SQLite e MySQL, seguindo uma arquitetura modular organizada em camadas (DAL/BLL) e uma metodologia de desenvolvimento baseada em Scrum. Serão ainda implementadas medidas de segurança para proteger credenciais, controlar permissões e garantir a integridade dos dados.

1. Enquadramento

O Sistema Integrado de Gestão para Servidores Discord de Roleplay é uma aplicação desenvolvida para automatizar processos administrativos de comunidades Roleplay.

A aplicação permitirá gerir tickets de suporte, candidaturas para organizações e equipas de staff, configuração do servidor, sistema de logs, gestão automática de cargos e integração direta com a base de dados do servidor de Roleplay.

Uma das principais funcionalidades será a sincronização automática entre o Discord e o servidor de jogo. Por exemplo, quando um jogador abandonar uma organização através de um painel no Discord, o sistema removerá automaticamente o cargo da organização, atualizará o emprego do jogador na base de dados, atribuirá o cargo **WhitelistBlock** e guardará esse estado na base de dados. Caso o utilizador abandone o Discord e volte posteriormente, o sistema verificará automaticamente esse registo e voltará a atribuir o cargo **WhitelistBlock**, garantindo que o estado permanece sincronizado.

Além disso, o projeto irá implementar mecanismos de segurança para proteger informações sensíveis da aplicação, assegurando que credenciais e configurações críticas não fiquem expostas.

O objetivo é reduzir tarefas repetitivas da equipa administrativa, diminuir erros humanos e centralizar toda a administração numa única plataforma.

2. Funcionalidades mínimas

- Sistema de tickets configurável.
- Sistema de candidaturas para organizações.
- Sistema de candidaturas para equipa de staff.
- Sistema de logs automáticos.
- Configuração de categorias e cargos através de comandos.
- Sistema de autorole.
- Sistema de **WhitelistBlock** persistente.
- Gestão automática das organizações.
- Remoção automática do cargo de uma organização através de um painel.
- Atualização automática da base de dados do servidor.
- Sincronização entre Discord e servidor de jogo.
- Geração de transcrições dos tickets.
- Implementação de medidas de segurança para proteção de credenciais e controlo de permissões.

3. Tecnologias utilizadas

Backend

- Python 3
- discord.py

Bases de Dados

- SQLite
- MySQL

Bibliotecas

- discord.py
- sqlite3
- mysql-connector-python
- python-dotenv
- asyncio
- io
- cryptography (para proteção de informação sensível)

Ferramentas

- Visual Studio Code
- Git
- GitHub

4. Estrutura da Base de Dados

SQLite

Tabela settings

- guild_id
- autorole_id
- logs_channel_id
- ticket_category_id
- staff_role_id

Tabela ticket_settings

- guild_id
- ticket_type
- category_id
- staff_role_id

Tabela whl_settings

- guild_id
- whl_type
- category_id
- staff_role_id

Tabela blocked_players

- discord_id
- license
- organization
- blocked
- blocked_at

MySQL (Servidor RP)

Tabela users

- identifier
- firstname
- lastname
- job
- job_grade

Outras tabelas

- jobs
- job_grades
- organizations

5. Simulação de Scrum

O desenvolvimento será dividido em cinco sprints.

Sprint 1

- Preparação do ambiente.
- Estrutura do projeto.
- Implementação da arquitetura DAL/BLL.
- Criação da base de dados SQLite.

Sprint 2

- Sistema de tickets.
- Sistema de logs.
- Configuração através de comandos.

Sprint 3

- Sistema de candidaturas.
- Sistema de whitelist.
- Transcrições automáticas.

Sprint 4

- Integração com MySQL.
- Comunicação com a base de dados do servidor.
- Gestão automática das organizações.
- Sistema **WhitelistBlock** persistente.
- Implementação das medidas de segurança.

Sprint 5

- Testes.
- Otimização.
- Correção de erros.
- Documentação.
- Apresentação final.

6. Funcionalidades futuras

- Integração completa com FiveM.
- Sistema automático de **SetJob**.
- Promoção e despromoção automática nas organizações.
- Histórico de organizações.
- Histórico administrativo.
- Dashboard de estatísticas.
- Website para gestão do bot.
- Painel administrativo online.
- Sistema de notificações para Discord e servidor.
- API própria para comunicação entre o bot e outros serviços.
- Sistema avançado de auditoria das ações administrativas.
- Implementação de autenticação para o painel de administração.

7. Medidas de Segurança

O projeto irá implementar um conjunto de medidas para proteger a aplicação e os dados armazenados:

- Armazenamento do token do Discord num ficheiro `.env`, evitando que fique visível no código-fonte.
- Utilização do ficheiro `.gitignore` para impedir a publicação de credenciais em repositórios Git.
- Possibilidade de encriptar informações sensíveis armazenadas pela aplicação.
- Validação dos dados recebidos antes de serem processados.
- Controlo de permissões para garantir que apenas utilizadores autorizados executam comandos administrativos.
- Registo de ações críticas em logs para auditoria e rastreabilidade.
- Separação da lógica de negócio (BLL) da camada de acesso aos dados (DAL), reduzindo o risco de alterações indevidas e facilitando a manutenção do código.
- Utilização de consultas parametrizadas nas bases de dados para minimizar o risco de ataques como **SQL Injection**.